Puppet Expert Seminar

Day 2

Advanced Puppet Language, Hiera, Module Design, Testing, and Code Quality

Day 2 — Agenda

Part 8 — Advanced Puppet Language

- Virtual and exported resources
- Advanced data types and type aliases
- Conditional logic patterns
- Iteration and data transformations

Part 9 — Hiera for Advanced Data Management

- Hiera v5 configuration
- Lookup strategies and merge behavior
- EYAML for encrypted secrets
- Designing a maintainable enterprise Hiera hierarchy

Part 10 — Professional Puppet Module Design

- Module structure and design principles
- Public vs. private classes
- Class parameter design
- Template management

Part 11 — Puppet Toolchain

- PDK, puppet-lint, editor support

Part 12 — Testing Puppet Code

- rspec-puppet unit testing
- Acceptance testing concepts
- CI/CD integration

Part 13 — Code Quality, Maintainability, and Collaboration

- Puppet code style
- Collaboration workflows
- Documentation standards

Part 8 — Advanced Puppet Language Concepts

Virtual Resources — The Problem They Solve

Consider managing users across multiple modules:

```
# Without virtual resources — this FAILS if both classes are included
class profile::webdev {
  user { 'deploy':
    ensure => present,
    uid    => 2001,
  }
}

class profile::appserver {
  user { 'deploy':            # Duplicate resource declaration — CATALOG ERROR
    ensure => present,
    uid    => 2001,
  }
}
```

Virtual resources decouple declaration from realization:

```
# Declare once — doesn't appear in catalog yet
@user { 'deploy':
  ensure => present,
  uid    => 2001,
  groups => ['www-data'],
}

# Realize in each class that needs it — safe to call multiple times
User <| title == 'deploy' |>
```

Virtual Resources — Patterns and Best Practices

```
# users module — declare all company users virtually
class users {
  @user { 'alice':   ensure => present, uid => 1001, groups => ['ops'] }
  @user { 'bob':     ensure => present, uid => 1002, groups => ['dev'] }
  @user { 'charlie': ensure => present, uid => 1003, groups => ['ops', 'dev'] }
  @user { 'svcacct': ensure => present, uid => 2001, groups => ['www-data'] }
}

# Any profile can now safely realize users it needs
class profile::webserver {
  include users    # activates virtual declarations

  # Realize specific users by attribute
  User <| groups == 'ops' or title == 'svcacct' |>
}

# Realize by title using the 'spaceship' operator
realize User['alice'], User['charlie']
```

Rules for virtual resources:

- Declare in a single place (usually a dedicated `users` or `resources` module)
- Realize only what a class actually needs
- Do NOT overuse — complex `<| |>` expressions are hard to debug
- Virtual resources are node-local — not shared with other nodes

Exported Resources — Cross-Node Awareness

Exported resources allow nodes to **share information through PuppetDB**:

```
# On each monitored node — EXPORT the monitoring check
@@nagios_service { "${facts['networking']['fqdn']}_http":
  host_name           => $facts['networking']['fqdn'],
  service_description => 'HTTP check',
  check_command       => 'check_http!80',
  use                 => 'generic-service',
  notify              => Service['nagios'],
}

# On the monitoring server — COLLECT all exported checks
Nagios_service <<| |>>
```

```
# Classic use case: SSH known hosts
# On every node — export its SSH host key
@@sshkey { $facts['networking']['fqdn']:
  type => 'ecdsa-sha2-nistp256',
  key  => $facts['ssh']['ecdsa']['key'],
}

# On every node — collect all known hosts
Sshkey <<| |>>
```

Exported Resources — Caution

Risk	Mitigation
PuppetDB query on every compile	Every `<<
Stale data	Decommissioned nodes may still have data in PDB
Security	Any node can export resources that affect other nodes
Ordering problems	Collected resources may conflict or duplicate

```
# Safer: use tags to limit collection scope
@@file { "/etc/haproxy/conf.d/${facts['networking']['fqdn']}.cfg":
  tag     => ['haproxy-backend', $datacenter],
  content => template('profile/haproxy_backend.erb'),
}

# Only collect from nodes in the same datacenter
File <<| tag == 'haproxy-backend' and tag == $datacenter |>>
```

> **Prefer other patterns over exported resources where possible.** A Hiera hash of backends is often simpler and more predictable.

Exported Resources — Use vs. Avoid

Situation	Exported resources?	Better alternative
HAProxy backends populated from web nodes	✅ Reasonable	Hiera hash + `create_resources` (simpler, no PDB dependency)
SSH `known_hosts` for all nodes	✅ Classic use case	—
Nagios host monitoring auto-registration	✅ Classic use case	—
Generating config files on >50 nodes	❌ Avoid	Hiera array + template iteration

Situation	Exported resources?	Better alternative
Firewall rules from dozens of sources	❌ Avoid	Central Hiera data structure
Any compile-time-critical environment	❌ Avoid	Exported resources add a PuppetDB roundtrip per `<<| |>>`
Large estates (>500 nodes, frequent changes)	❌ Avoid	PuppetDB write storms degrade catalog performance

Performance rule of thumb: Each `<<| |>>` collection forces a synchronous PuppetDB query during catalog compilation. Ten collections on a node with 1,000 peers = 10 PDB queries per compile.

Advanced Data Types

Puppet's type system allows precise API contracts for module parameters:

```
# Basic scalar types
String $name                        # any string
Integer $port                       # integer
Float $threshold                    # floating-point
Boolean $enabled                    # true/false
Array $items                        # untyped array
Hash $config                        # untyped hash

# Constrained types
String[1]           $nonempty       # string with minimum length 1
String[1, 64]       $hostname       # string between 1 and 64 chars
Integer[1, 65535]   $port           # integer in valid port range
Integer[0]          $nonneg         # non-negative integer
Float[0.0, 1.0]     $ratio          # float between 0 and 1
Array[String]       $names          # array of strings only
Hash[String, Integer] $scores       # hash with string keys, integer values
```

Optional, Variant, Enum, and Struct Types

```
# Optional — value can be the type OR undef
Optional[String]    $comment      # string or undef
Optional[Integer]   $timeout      # integer or undef (use default when undef)

# Variant — value can be one of several types
Variant[String, Integer, Boolean] $flexible_value

# Enum — value must be one of these exact strings
Enum['present', 'absent', 'latest'] $ensure

# Pattern — string must match a regex
Pattern[/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/] $ip_address

# Struct — strongly typed hash with defined keys
Struct[{
  host    => String[1],
  port    => Integer[1, 65535],
  ssl     => Boolean,
  timeout => Optional[Integer[1]],
}] $db_connection
```

Sensitive Type — Protecting Secrets

```
# In your class definition
class myapp::database (
  String[1]                $db_host,
  Integer[1, 65535]        $db_port     = 5432,
  Sensitive[String[1]]     $db_password,    # Puppet will NOT log this value
) {
  file { '/etc/myapp/database.conf':
    owner   => 'myapp',
    mode    => '0640',
    content => epp('myapp/database.conf.epp', {
      host     => $db_host,
      port     => $db_port,
      password => $db_password.unwrap,   # unwrap inside template call only
    }),
  }
}
```

```
# In Hiera — store via EYAML
myapp::database::db_password: >
  ENC[PKCS7,MIIBeQYJKo...]
```

> Use `Sensitive` for any parameter that holds a password, token, or private key. Puppet will redact it from logs and reports.

Type Aliases — Reusable Type Definitions

```
# In lib/puppet/type_aliases/myorg/port.pp
type Myorg::Port = Integer[1, 65535]

# In lib/puppet/type_aliases/myorg/db_config.pp
type Myorg::DbConfig = Struct[{
  host     => String[1],
  port     => Myorg::Port,
  database => String[1],
  username => String[1],
  password => Sensitive[String[1]],
}]

# Usage in any class
class myapp::database (
  Myorg::DbConfig $connection,
) {
  # ...
}
```

Iteration Patterns — Data-Driven Resource Creation

```
# Create resources from a Hiera hash
# In Hiera:
#   myapp::vhosts:
#     shop.example.com:
#       port: 443
#       ssl:  true
#       root: /var/www/shop
#     blog.example.com:
#       port: 80
#       ssl:  false
#       root: /var/www/blog

class myapp (
  Hash[String, Hash] $vhosts = {},
) {
  $vhosts.each |String $name, Hash $config| {
    nginx::resource::server { $name:
      listen_port => $config['port'],
      ssl         => $config['ssl'],
      www_root    => $config['root'],
    }
  }
}
```

Advanced Iteration — map, filter, reduce

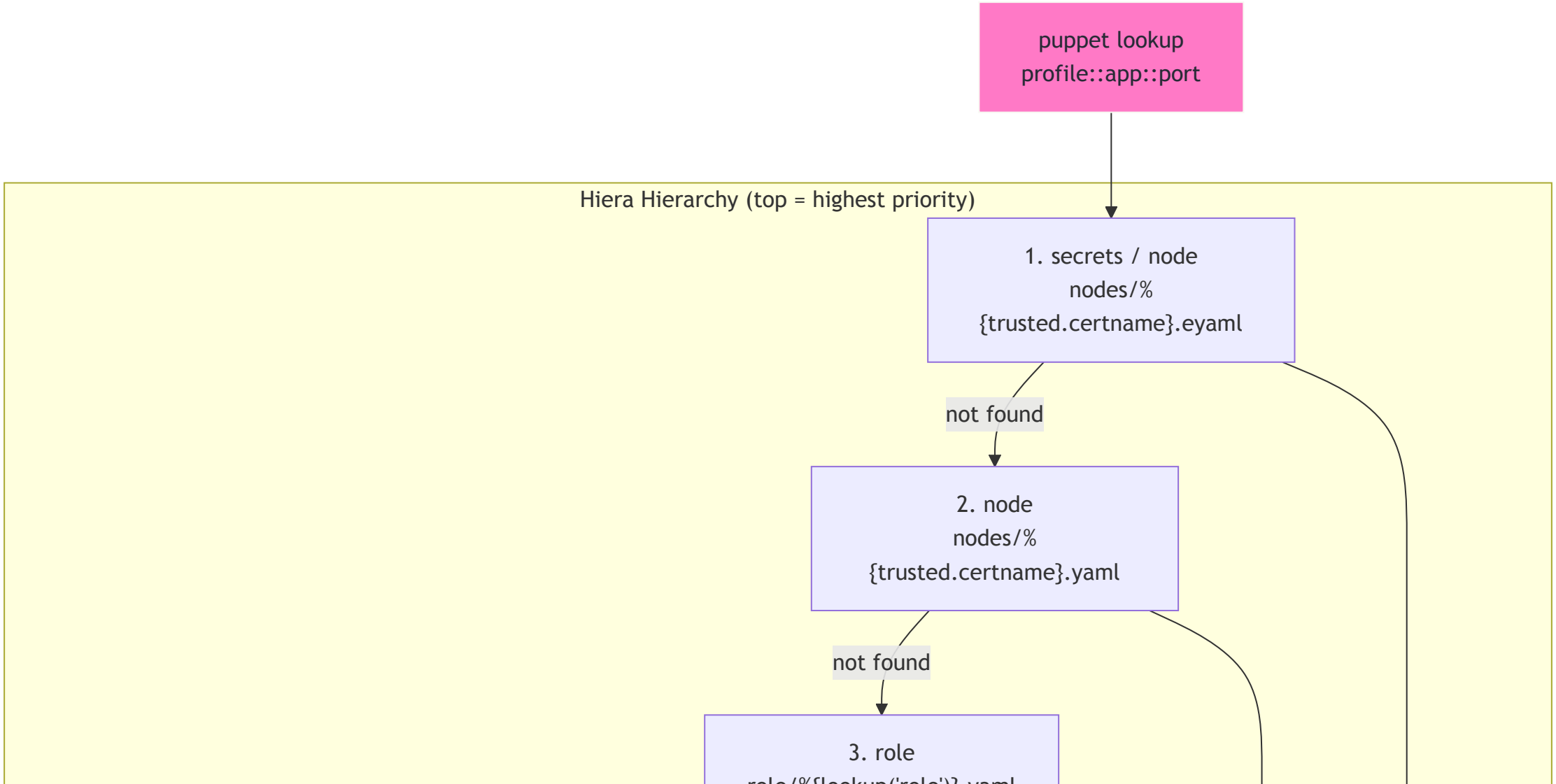
```
# map: transform data before using it
$raw_users = lookup('ops::users', Array[Hash])   # [{name:alice, uid:1001}, ...]

$user_names = $raw_users.map |$u| { $u['name'] }
# → ['alice', 'bob', 'charlie']

$uid_map = $raw_users.reduce({}) |$acc, $u| {
  $acc + { $u['name'] => $u['uid'] }
}
# → {alice => 1001, bob => 1002, charlie => 1003}

# filter: select subset
$admin_users = $raw_users.filter |$u| { $u['role'] == 'admin' }

# Combine: create user resources only for admins on this system
$admin_users.each |$user| {
  user { $user['name']:
    ensure => present,
    uid    => $user['uid'],
    groups => ['sudo'],
  }
}
```

Part 9 — Hiera for Advanced Data Management

Hiera v5 — Configuration Structure

```yaml
# /etc/puppetlabs/code/environments/production/hiera.yaml
---
version: 5

defaults:
  datadir: "data"
  data_hash: yaml_data

hierarchy:
  - name: "Secrets (EYAML)"
    lookup_key: eyaml_lookup_key
    datadir: "secret-data"
    paths:
      - "nodes/%{trusted.certname}.eyaml"
      - "roles/%{lookup('role')}.eyaml"
      - "common.eyaml"
    options:
      pkcs7_private_key: /etc/puppetlabs/puppet/keys/private_key.pkcs7.pem
      pkcs7_public_key:  /etc/puppetlabs/puppet/keys/public_key.pkcs7.pem

  - name: "Node-specific data"
    paths:
      - "nodes/%{trusted.certname}.yaml"

  - name: "Role data"
    paths:
      - "roles/%{lookup('role')}.yaml"

  - name: "Profile data"
    paths:
      - "profiles/%{lookup('profile')}.yaml"

  - name: "OS + version"
    paths:
      - "os/%{facts.os.name}-%{facts.os.release.major}.yaml"
      - "os/%{facts.os.family}.yaml"

  - name: "Datacenter"
    paths:
      - "dc/%{facts.datacenter}.yaml"

  - name: "Environment"
    paths:
      - "env/%{environment}.yaml"

  - name: "Common"
    path: "common.yaml"
```

The Advanced Hiera Hierarchy



Lookup Strategies — Deep Merge

```
# --- in Hiera data ---
# common.yaml
profile::base::packages:
  - vim
  - curl
  - htop

# os/RedHat.yaml
profile::base::packages:
  - dnf-plugins-core
  - policycoreutils-python-utils

# nodes/web01.example.com.yaml
profile::base::packages:
  - tcpdump
  - strace

# --- In Puppet manifest or hiera.yaml hierarchy lookup ---
# With lookup strategy 'first' (default):
#   → ['tcpdump', 'strace']   (only node level, stops there)

# With lookup strategy 'unique' (array merge):
#   → ['vim', 'curl', 'htop', 'dnf-plugins-core', 'policycoreutils-python-utils', 'tcpdump', 'strace']
#   (combines ALL matching arrays, deduplicates)

# Usage in class:
class profile::base (
  Array[String] $packages = lookup('profile::base::packages', {merge => unique, default_value => []}),
) {
  package { $packages: ensure => installed }
}
```

Hash Deep Merge

```yaml
# common.yaml
profile::nginx::config:
  worker_processes: 4
  keepalive_timeout: 65
  gzip: true
  log_format: combined

# env/production.yaml
profile::nginx::config:
  worker_processes: 8
  client_max_body_size: 100m

# nodes/bigweb01.yaml
profile::nginx::config:
  worker_processes: 16
```

```
# Deep merge — all keys merged, higher-priority wins on conflict
$config = lookup('profile::nginx::config', {
  merge => {
```

Debugging Hiera Lookups

```
# Explain why a value was found (most useful command for Hiera debugging)
puppet lookup profile::nginx::worker_processes \
  --node web01.example.com \
  --environment production \
  --explain

# Output shows every hierarchy level checked:
# Hierarchy entry "Node-specific data"
#   Path "data/nodes/web01.example.com.yaml": key not found
# Hierarchy entry "Role data"
#   Path "data/roles/webserver.yaml": value found → 8

# Preview the merged hash
puppet lookup profile::nginx::config \
  --node web01.example.com \
  --environment production \
  --merge deep \
  --render-as json
```

Hiera EYAML — Encrypted Secrets

```
# Install the gem (inside Puppet Server / agent Ruby)
/opt/puppetlabs/puppet/bin/gem install hiera-eyaml

# Generate encryption keys (run once, store keys securely)
eyaml createkeys
# Creates: keys/public_key.pkcs7.pem  ← share with build servers
#          keys/private_key.pkcs7.pem  ← NEVER commit to Git

# Encrypt a value
eyaml encrypt -s 'SuperSecretPassword123'
# Output: ENC[PKCS7,MIIBeQYJKoZI...]

# Encrypt a file
eyaml encrypt -f /path/to/secret.txt
```

```yaml
# data/common.eyaml
---
myapp::database::password: ENC[PKCS7,MIIBeQYJKoZIhvcNAQcDoIIBKjCCASYCAQAxggEf...]
myapp::api::token: ENC[PKCS7,MIIBeQYJKo...]
```

Key management rules:

- Private key must be on Puppet Server only — never in Git
- Rotate keys using `eyaml recrypt`
- Backup private key in a secure vault (HashiCorp Vault, AWS Secrets Manager, etc.)
- Give build/CI servers the public key only (they can encrypt, not decrypt)

Designing a Production Hiera Hierarchy

```
data/
├── secret-data/           ← EYAML files, gitignored or separate repo
│   ├── common.eyaml
│   └── nodes/
├── nodes/                 ← Node-specific overrides (use sparingly)
│   └── special-node.example.com.yaml
├── roles/                 ← Role-level parameters
│   ├── webserver.yaml
│   ├── database.yaml
│   ├── monitor.yaml
│   └── base.yaml
├── profiles/              ← Profile-level parameters (fine-grained)
│   ├── web__nginx.yaml
│   └── base__ntp.yaml
├── os/                    ← OS-specific defaults
│   ├── RedHat-9.yaml
│   ├── RedHat-8.yaml
│   ├── Ubuntu-22.04.yaml
│   └── Debian.yaml
├── dc/                    ← Datacenter-specific parameters
│   ├── ams01.yaml
│   └── fra01.yaml
├── env/                   ← Environment-specific overrides
│   ├── production.yaml
│   └── staging.yaml
└── common.yaml            ← Global defaults — the baseline
```

Hiera Anti-Patterns to Avoid

Anti-pattern	What goes wrong	Fix
Everything in `nodes/<hostname>.yaml`	Hierachical data advantage lost; hard to refactor	Push data up to `roles/` or `os/` layers
Duplicate keys at multiple levels without `lookup_options`	First-found wins silently; overrides are invisible	Use `merge: deep` or document override intent
Sensitive values in plain YAML	Secrets in Git history permanently	Use EYAML for any credential or key
Giant `common.yaml` (>500 lines)	Unmaintainable; unrelated concerns mixed	Split into `common/` directory with per-technology files

Part 10 — Professional Puppet Module Design

Module Design Principles — Expert Level

Principle 1: Single responsibility

Each module manages exactly one technology. `puppet-nginx` manages nginx. It does NOT manage PHP, certificates, or monitoring.

Principle 2: Public API surface

Only `init.pp` (and explicitly documented classes) are the public API. Internal classes (`mymodule::install`, `mymodule::config`) are private — callers must not include them directly.

Principle 3: Data-code separation

A well-designed module has NO environment-specific data. All data lives in Hiera. The module provides only structure and behaviour.

Principle 4: Testable parameters

Every parameter should have a type. Every important code path should be testable by passing different parameter values — no internal conditionals that callers cannot influence.

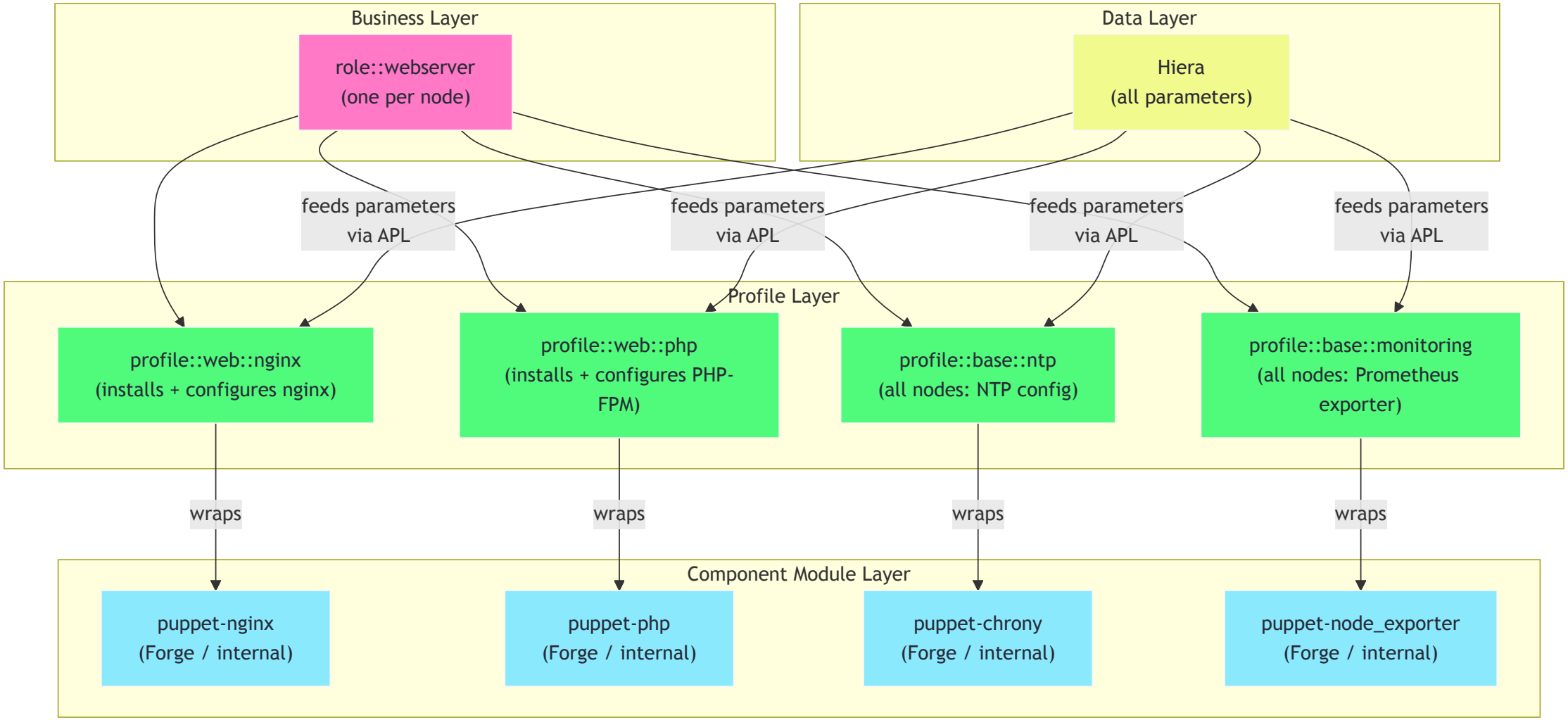
Principle 5: Idempotent, convergent behaviour

Every resource the module manages must be idempotent. The module must reach the same final state regardless of the starting state.

Professional Module Layout

```
puppet-nginx/
├── manifests/
│   ├── init.pp           ← Public: class nginx(...) — the only entry point
│   ├── install.pp        ← Private: class nginx::install
│   ├── config.pp         ← Private: class nginx::config
│   ├── service.pp        ← Private: class nginx::service
│   └── vhost.pp          ← Public defined type: nginx::vhost
├── templates/
│   ├── nginx.conf.epp    ← Main configuration file
│   └── vhost.conf.epp    ← Virtual host fragment
├── files/
│   └── mime.types        ← Static files served unchanged
├── lib/
│   ├── puppet/
│   │   └── functions/
│   │       └── nginx/
│   │           └── validate_vhost.rb    ← Custom function
│   └── facter/
│       └── nginx_version.rb             ← Custom fact
├── spec/
│   ├── classes/
│   │   └── init_spec.rb
│   └── defined_types/
│       └── vhost_spec.rb
├── data/
│   └── common.yaml       ← Module-level default data
├── hiera.yaml            ← Module-level Hiera config
├── metadata.json
├── CHANGELOG.md
└── README.md
```

The Roles and Profiles Pattern — Expert Depth



Module Parameter Design — Best Practices

Good parameter design makes a module usable without reading the source:

```
class nginx (
  # ✅ Typed, Optional with sensible default
  String[1]           $package_name    = 'nginx',
  Stdlib::Port        $port            = 80,
  Boolean             $manage_service  = true,
  Enum['running','stopped'] $service_ensure = 'running',

  # ✅ Optional — undef means "don't manage it"
  Optional[String[1]] $ssl_cert        = undef,
  Optional[String[1]] $ssl_key         = undef,

  # ✅ Typed hash for open-ended config — callers can add arbitrary keys
  Hash[String, Scalar] $extra_settings = {},

  # ❌ Avoid raw Hash or Array — callers get no type checking
  # Hash $config = {},
) {
  if $ssl_cert and !$ssl_key {
    fail("nginx: ssl_cert requires ssl_key to also be set")
  }
```

Rules:

- Use `Stdlib::*` types (from `puppetlabs-stdlib`) for ports, IPs, paths
- Use `Optional[T]` rather than defaulting to `''` or `'UNSET'`
- Validate cross-parameter constraints with `fail()` at the top of the class

Component Module: nginx — init.pp

```
# @summary Manages the nginx web server.
#
# @param package_name
#   Name of the nginx package to install. Defaults differ by OS.
# @param config_dir
#   Directory for nginx configuration files.
# @param worker_processes
#   Number of nginx worker processes. Use 'auto' to match CPU count.
# @param worker_connections
#   Maximum simultaneous connections per worker process.
# @param manage_service
#   Whether to manage the nginx service. Set false to manage externally.
#
class nginx (
  String[1]              $package_name      = 'nginx',
  Stdlib::Absolutepath   $config_dir        = '/etc/nginx',
  Variant[Integer[1], Enum['auto']] $worker_processes = 'auto',
  Integer[128, 65535]    $worker_connections = 1024,
  Boolean                $manage_service    = true,
  Enum['running','stopped'] $service_ensure = 'running',
  Boolean                $service_enable    = true,
) {
  contain nginx::install
  contain nginx::config
  contain nginx::service

  Class['nginx::install']
  -> Class['nginx::config']
  ~> Class['nginx::service']
}
```

Component Module: nginx — install.pp, config.pp

```
# @api private
class nginx::install {
  package { $nginx::package_name:
    ensure => installed,
  }
}
```

```
# @api private
class nginx::config {
  file { $nginx::config_dir:
    ensure => directory,
    owner  => 'root',
    group  => 'root',
    mode   => '0755',
  }

  file { "${nginx::config_dir}/nginx.conf":
    ensure  => file,
    owner   => 'root',
    group   => 'root',
    mode    => '0644',
    content => epp('nginx/nginx.conf.epp', {
      worker_processes   => $nginx::worker_processes,
      worker_connections => $nginx::worker_connections,
      config_dir         => $nginx::config_dir,
    }),
    require => File[$nginx::config_dir],
  }
}
```

Defined Type: nginx::vhost

```
# @summary Manage a single nginx virtual host.
#
# @param server_name
#   List of server names for this virtual host.
# @param listen_port
#   TCP port to listen on.
# @param root
#   Document root directory.
# @param ssl
#   Whether to enable SSL/TLS.
# @param ssl_cert
#   Path to the SSL certificate file.
# @param ssl_key
#   Path to the SSL private key file.
#
define nginx::vhost (
  Array[String[1], 1]    $server_name  = [$name],
  Integer[1, 65535]      $listen_port  = 80,
  Optional[Stdlib::Absolutepath] $root = undef,
  Boolean                $ssl          = false,
  Optional[Stdlib::Absolutepath] $ssl_cert = undef,
  Optional[Stdlib::Absolutepath] $ssl_key  = undef,
) {
  if $ssl and ($ssl_cert == undef or $ssl_key == undef) {
    fail("nginx::vhost[${name}]: ssl_cert and ssl_key are required when ssl => true")
  }

  file { "${nginx::config_dir}/conf.d/${name}.conf":
    ensure  => file,
    owner   => 'root',
    group   => 'root',
    mode    => '0644',
    content => epp('nginx/vhost.conf.epp', {
      name        => $name,
      server_name => $server_name,
      listen_port => $listen_port,
      root        => $root,
      ssl         => $ssl,
      ssl_cert    => $ssl_cert,
      ssl_key     => $ssl_key,
    }),
    notify  => Service['nginx'],
  }
}
```

EPP Templates — Expert Patterns

```
<%- | String $name,
      Array[String] $server_name,
      Integer $listen_port,
      Optional[Stdlib::Absolutepath] $root,
      Boolean $ssl,
      Optional[Stdlib::Absolutepath] $ssl_cert,
      Optional[Stdlib::Absolutepath] $ssl_key,
| -%>
# Managed by Puppet - DO NOT EDIT
# Module: nginx, Defined type: nginx::vhost

server {
  listen      <%= $listen_port %><% if $ssl { %> ssl<% } %>;
  server_name <%= $server_name.join(' ') %>;

<% if $root { -%>
  root  <%= $root %>;
  index index.html index.php;
<% } -%>

<% if $ssl { -%>
  ssl_certificate     <%= $ssl_cert %>;
  ssl_certificate_key <%= $ssl_key %>;
  ssl_protocols       TLSv1.2 TLSv1.3;
  ssl_ciphers         HIGH:!aNULL:!MD5;
<% } -%>

  access_log /var/log/nginx/<%= $name %>-access.log;
  error_log  /var/log/nginx/<%= $name %>-error.log;
}
```

Part 11 — Puppet Toolchain

PDK — Puppet Development Kit

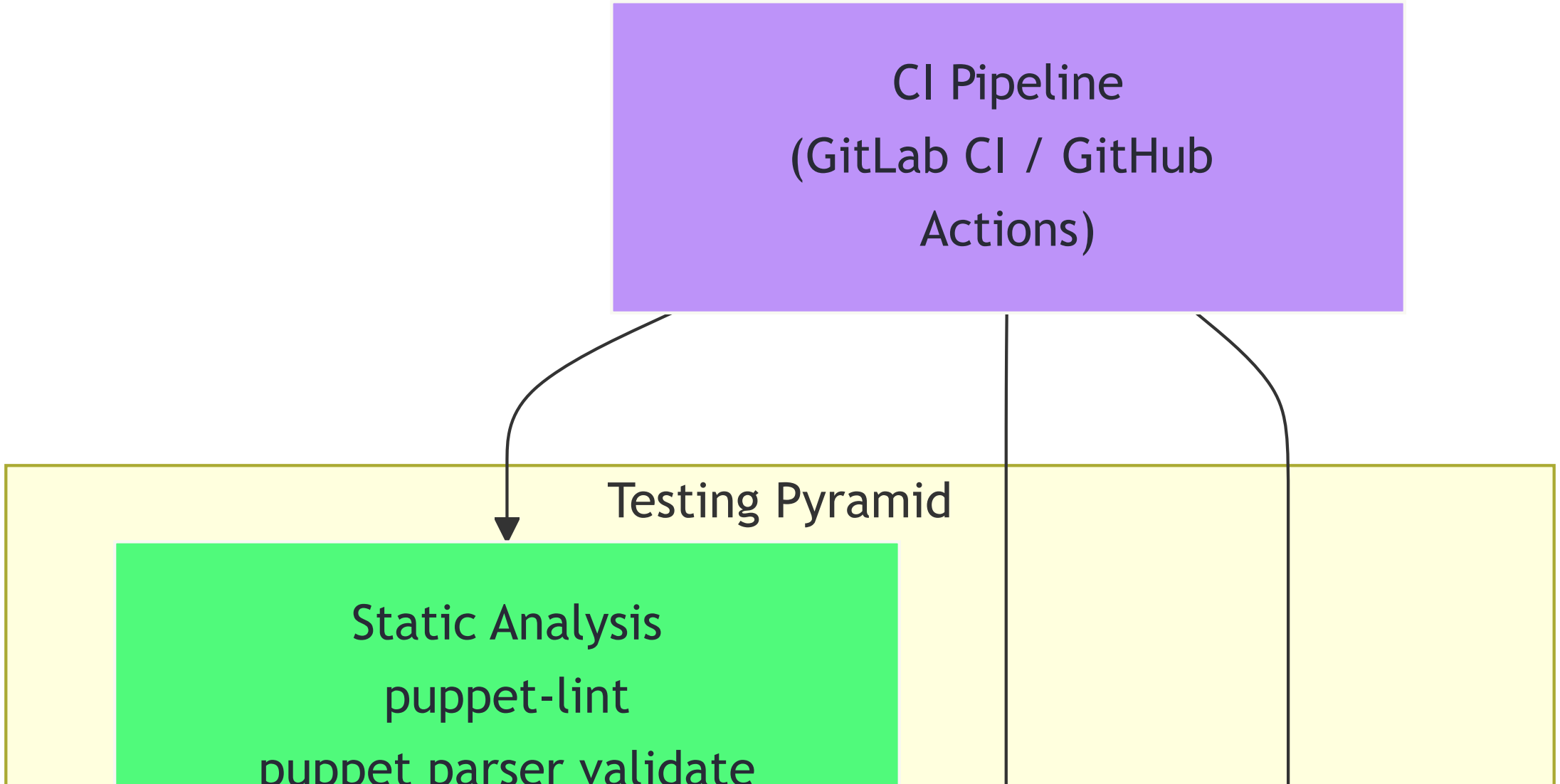
```
# Install PDK
# Ubuntu/Debian:
wget https://apt.puppet.com/puppet-tools-release-jammy.deb
dpkg -i puppet-tools-release-jammy.deb
apt-get install -y pdk

# RHEL/Rocky:
rpm -Uvh https://yum.puppet.com/puppet-tools-release-el-9.noarch.rpm
dnf install -y pdk

# Create a new module
pdk new module myorg-nginx
cd myorg-nginx

# Generate classes and defined types
pdk new class nginx           # creates manifests/init.pp + spec skeleton
pdk new class nginx::install
pdk new defined_type nginx::vhost

# Validate code (syntax + metadata + Rubocop)
pdk validate

# Run unit tests
pdk test unit

# Run unit tests for specific spec file
pdk test unit --tests spec/classes/init_spec.rb

# Upgrade PDK template to latest
pdk update
```

puppet-lint — Code Style Enforcement

```
# Run puppet-lint on a single file
puppet-lint manifests/init.pp

# Run on all manifests (with auto-fix for fixable issues)
puppet-lint --fix manifests/

# Configure lint checks (.puppet-lint.rc):
cat > .puppet-lint.rc << 'EOF'
--no-80chars-check
--no-documentation-check
--no-autoloader_layout-check
--relative
EOF

# Common issues reported:
# WARNING: two-space soft tabs not used on line X
# WARNING: trailing whitespace found
# WARNING: => aren't lined up
# ERROR: class inherits from params class (antipattern)
# WARNING: ensure found on line but not the first attribute

# Useful lint plugins (install as gems):
gem install puppet-lint-param-docs
gem install puppet-lint-strict_indent-check
gem install puppet-lint-trailing_newline-check
```

Part 12 — Testing Puppet Code

Testing Strategy



rspec-puppet — Unit Testing Philosophy

rspec-puppet tests that your **catalog compiles correctly** with specific inputs.

It does NOT test:

- That the code actually runs on a real system
- That resources have the correct effect on the OS
- Integration between multiple modules

It DOES test:

- That given these parameters and facts, these resources exist in the catalog
- That invalid parameters raise the correct errors
- That conditional logic produces the right resources on different OS families
- That resource attributes have the expected values

> Think of rspec-puppet as **contract testing for your module's API**.

rspec-puppet — Writing Tests

```ruby
# spec/classes/init_spec.rb
require 'spec_helper'

describe 'nginx' do
  # Test on supported OS families
  on_supported_os.each do |os, os_facts|
    context "on #{os}" do
      let(:facts) { os_facts }

      # Default parameters — catalog must compile
      context 'with default parameters' do
        it { is_expected.to compile.with_all_deps }
        it { is_expected.to contain_package('nginx').with_ensure('installed') }
        it { is_expected.to contain_service('nginx').with_ensure('running') }
        it { is_expected.to contain_class('nginx::install') }
        it { is_expected.to contain_class('nginx::config') }
        it { is_expected.to contain_class('nginx::service') }

        it 'installs before configuring' do
          is_expected.to contain_class('nginx::install')
            .that_comes_before('Class[nginx::config]')
        end
      end

      # Custom parameters
      context 'with manage_service => false' do
        let(:params) { { manage_service: false } }
        it { is_expected.not_to contain_service('nginx') }
      end

      # Type validation
      context 'with invalid worker_processes' do
        let(:params) { { worker_processes: 'invalid_string' } }
        it { is_expected.to compile.and_raise_error(/expects.*Integer.*Enum/) }
      end
    end
  end
end
```

rspec-puppet — Testing Defined Types

```ruby
# spec/defined_types/vhost_spec.rb
require 'spec_helper'

describe 'nginx::vhost' do
  let(:pre_condition) { 'include nginx' }

  context 'basic HTTP vhost' do
    let(:title) { 'mysite.example.com' }
    let(:params) do
      {
        listen_port: 80,
        root: '/var/www/mysite',
      }
    end

    it { is_expected.to compile.with_all_deps }
    it {
      is_expected.to contain_file('/etc/nginx/conf.d/mysite.example.com.conf')
        .with_ensure('file')
        .with_mode('0644')
        .with_content(matching(%r{server_name mysite\.example\.com}))
        .with_content(matching(%r{listen 80}))
        .with_content(matching(%r{root /var/www/mysite}))
    }
  end

  context 'SSL vhost without cert — should fail' do
    let(:title) { 'secure.example.com' }
    let(:params) { { ssl: true } }
    it { is_expected.to compile.and_raise_error(/ssl_cert and ssl_key are required/) }
  end

  context 'SSL vhost with cert' do
    let(:title) { 'secure.example.com' }
    let(:params) do
      {
        ssl: true,
        listen_port: 443,
        ssl_cert: '/etc/ssl/certs/secure.crt',
        ssl_key:  '/etc/ssl/private/secure.key',
      }
    end
    it { is_expected.to compile.with_all_deps }
    it {
      is_expected.to contain_file('/etc/nginx/conf.d/secure.example.com.conf')
        .with_content(matching(%r{ssl_certificate /etc/ssl/certs/secure\.crt}))
    }
  end
end
```

rspec-puppet — Testing with Hiera Data

```ruby
# spec/classes/init_spec.rb — testing with Hiera
describe 'profile::webserver' do
  let(:hiera_config) { 'spec/fixtures/hiera/hiera.yaml' }

  on_supported_os.each do |os, os_facts|
    context "on #{os}" do
      let(:facts) { os_facts.merge({ datacenter: 'ams01' }) }

      context 'with production Hiera data' do
        let(:node_params) { { environment: 'production' } }

        it { is_expected.to compile.with_all_deps }
        it { is_expected.to contain_nginx__vhost('shop.example.com') }
      end
    end
  end
end
```

```yaml
# spec/fixtures/hiera/hiera.yaml
---
version: 5
hierarchy:
  - name: "Test data"
    path: "%{facts.os.family}.yaml"
  - name: "Common"
    path: "common.yaml"
defaults:
  datadir: spec/fixtures/hiera/data
```

CI/CD Pipeline — Fully Integrated

```yaml
# .gitlab-ci.yml — complete Puppet module pipeline
stages: [validate, test, release]

.puppet_base: &puppet_base
  image: puppet/pdk:latest
  before_script:
    - pdk bundle install

syntax-check:
  <<: *puppet_base
  stage: validate
  script:
    - pdk validate metadata
    - pdk validate puppet
    - pdk validate ruby

lint:
  <<: *puppet_base
  stage: validate
  script:
    - pdk bundle exec puppet-lint --fail-on-warnings manifests/

unit-tests:
  <<: *puppet_base
  stage: test
  script:
    - pdk test unit --format junit --format progress
  artifacts:
    reports:
      junit: junit_output.xml

acceptance-tests:
  stage: test
  script:
    - pdk bundle exec rake litmus:provision_list[docker]
    - pdk bundle exec rake litmus:install_agent
    - pdk bundle exec rake litmus:install_module
    - pdk bundle exec rake litmus:acceptance:parallel
  when: manual
  allow_failure: true
```

Part 13 — Code Quality, Maintainability, and Collaboration

Puppet Code Style — Naming Conventions

```
# Class names: lowercase, underscores, module::class hierarchy
class myorg_nginx {}          # WRONG – no module prefix
class Nginx::Config {}        # WRONG – uppercase
class nginx::config {}        # CORRECT

# Variable names: lowercase, underscores
$MyVariable = 'value'         # WRONG
$my_variable = 'value'        # CORRECT

# Resource titles: consistent, predictable (often the file path)
File { 'etc nginx conf':  ... }         # WRONG – spaces in title
file { '/etc/nginx/nginx.conf': ... }   # CORRECT

# Parameter alignment: arrows aligned per resource block
file { '/etc/nginx/nginx.conf':
  ensure  => file,
  owner   => 'root',       # CORRECT – aligned
  mode    => '0644',
  content => ...,
}
```

Code Review for Puppet — Checklist

Correctness:

- [ ] Does the catalog compile without errors?
- [ ] Are all resources idempotent?
- [ ] Are dependency relationships correct (no cycles, no missing `require`)?
- [ ] Are all `exec` resources guarded with `onlyif`, `unless`, or `creates`?

Safety:

- [ ] No secrets hardcoded in manifests or templates
- [ ] No `exec` with untrusted/user-controlled strings (command injection risk)
- [ ] No world-readable files containing sensitive content
- [ ] Autosigning disabled or policy-based only

Maintainability:

Documentation Standards

```
# Every public class and defined type must have a @summary and @param docs

# @summary Manages the nginx web server and virtual host configuration.
#
# @example Basic usage
#   include nginx
#
# @example With custom worker count
#   class { 'nginx':
#     worker_processes => 8,
#   }
#
# @param worker_processes
#   Number of worker processes to start. Set to 'auto' to match
#   the number of available CPU cores. Default: 'auto'.
#
# @param manage_service
#   When false, Puppet will not manage the nginx service state.
#   Useful when service management is handled by an external tool.
#   Default: true.
#
class nginx (
  Variant[Integer[1], Enum['auto']] $worker_processes = 'auto',
  Boolean                           $manage_service   = true,
) {
```

Day 2 — Summary

Language mastery:

- Virtual and exported resources for cross-class and cross-node coordination
- Full type system usage including `Sensitive`, `Struct`, and type aliases
- Data-driven resource creation with iteration

Data mastery:

- Hiera v5 multi-level hierarchy design
- Deep merge strategies for array and hash data
- EYAML for encrypted secrets at rest

Module and code quality mastery:

- Professional module layout with public/private class discipline
- Typed parameters and EPP templates
- rspec-puppet unit testing at expert level
- Code review criteria for production Puppet code

Tomorrow — Day 3:

Ruby for Puppet developers, custom functions, facts, types and providers, Bolt, security, performance, Foreman, and enterprise best practices.

Day 2 — Practical Exercises

Exercise 1: Advanced Puppet Language — Virtual Resources, Data Types, and Iteration

Exercise 2: Hiera Deep Merge, EYAML Secrets, Module Design, and rspec-puppet Tests